

1 Lesson 01-10-26

1.1 Freestanding C

The C standard defines two kinds of environment:

- **Hosted:** C library (`libc`) and an OS sit under the user code, library functions such as `printf()`, `malloc()`, `fopen()` are available. `main()` is called by the `libc` startup code, called by the OS. Of course, each OS implementation implements its own `libc` (GNU/Linux, Windows, ...). There are even versions of `libc` which run on bare metal (`newlib`);
- **Freestanding:** nothing sits under the user code, no library functions are available and nobody calls `main()` (the default entry point is the reset handler).

We will be using a *freestanding* environment from now on.

System calls

In fact, `libc` in Linux is only an interface between user code and the OS: a call to a `libc` function in Linux means:

- Performing a little setup work in userland, via the `libc` implementations;
- Performing some kind of *system call*, which leaves the CPU to the kernel;
- Waiting for the kernel to carry out our system call;
- Getting the result back from `libc`.

We decide to opt out of libraries like `libc` because they still need architecture-specific hooks (`write`, `crt0`, ...) hooks to be implemented: we're better off doing everything ourselves.

Compiler helpers

One piece of help we will use is the *compiler helpers* provided by the **GCC** (*Gnu Compiler Collection*) toolchain we use. These implement some basic C operators (such as divide and modulo), which may not be implemented in hardware. Was someone to be curious, some example helpers can be found at <https://github.com/seggiani-luca/micro-sim/blob/main/eprom/src/lib/helpers.cpp> (there are likely cleaner and better implemented one somewhere in the GCC source).

1.1.1 Semihosting

We still need to be able to print stuff to console. The solution we will use is **semihosting**: the target asks the *host* (our machine) to perform the I/O.

1. The program puts an operation code in `r0` (`0x04 = SYS_WRITE0`) and the string address in `r1`. `SYS_WRITE0` is the opcode for "print this NULL-terminated string";
2. It executes `bkpt 0xab`: a breakpoint instruction with a special number;
3. QEMU (or a debugger) sees the breakpoint, reads `r0` and `r1`, reads the string from target memory, and prints it **on your laptop**;
4. The program resumes after the `bkpt`.

This only works with a debugger or emulator attached (e.g., `-semihosting` in `make run`). On a real board without a debugger, `bkpt` would simply stop the CPU.

In code

In code, this translates to:

```
1 static void semihost_write0( const char *message )
2 {
3     register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
4     register const char *r1 __asm__( "r1" ) = message; /* string address
5     /*
6     __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
```

Let's look at what `bkpt 0xab` really does:

- The program **stops** doing its own work;
- Control passes to **someone more powerful** (QEMU, outside the CPU);
- That someone performs the job and **hands control back**.

This is the same shape as a system call on Linux: your program cannot drive the terminal itself, so it **asks the kernel** to do it.

Next question: can the Cortex-M3 itself have a 'powerful' part and a 'restricted' part, like kernel mode and user mode? **Yes**: privileged and unprivileged execution, next slide.

1.2 Privilege level

Even if our implementation of an OS will be *flat*, let's go over **privilege** levels in our ARM Cortex-M3 CPU.

We know that an *OS kernel* runs in kernel mode, whereas an application runs in *user mode*. Cortex-M3 has the same idea in hardware:

- **Handler mode**: used when running an exception handler, always privileged;
- **Thread mode**: normal mode used for user code, can be privileged or unprivileged.

After reset we find the CPU in thread mode, privileged. We have to manually set the CPU to unprivileged thread mode if we want protection on user code.

A special CPU register (not in memory) selects Thread-mode privilege:

Bit	Name	Meaning
0	nPRIV	0 = privileged, 1 = unprivileged (Thread mode only)
1	SPSEL	selects which stack pointer Thread mode uses (not used here)

Special registers are reached with two dedicated instructions:

Instruction	C equivalent
<code>mrs r0, control</code>	<code>r0 = CONTROL;</code> (read special register)
<code>msr control, r0</code>	<code>CONTROL = r0;</code> (write special register)

1.2.1 Dropping privilege

So, for example, to drop our privilege lever to unprivileged:

```

1 mrs r0, control      @ r0 = CONTROL
2 orr r0, r0, #1       @ r0 = r0 | 1      (set nPRIV)
3 msr control, r0      @ CONTROL = r0     -> now unprivileged
4 isb                  @ discard instructions already fetched under old rules

```

Writing CONTROL is itself a privileged operation: an unprivileged `msr control, ...` is ignored. Code cannot promote itself back. The only ways back to privileged: reset, or an exception (its handler runs in Handler mode, always privileged).

Attempt	Result
Read/write the System Control Space (0xE000E000 ...): SysTick, NVIC, SCB (VTOR, SHCSR, CCR, priorities)	BusFault
<code>msr control, r0</code> (raise privilege back)	ignored
<code>cpsid i / cpsie i</code> (mask interrupts)	ignored
Semihosting bkpt 0xab	QEMU refuses it by default → fault

Ordinary RAM, Flash, and user code still work normally. The restricted code simply cannot **reconfigure** the machine or silence it.

1.2.2 Raising privilege

Unprivileged code cannot touch the hardware, but it can still *call* privileged mode handlers: `svc #n` (*SuperVisor Call*) is allowed in unprivileged code.

1. It raises the **SVC** **exception** (vector-table entry 11, `SVC_Handler`). The CPU enters **Handler mode**, and the corresponding handler runs privileged;
2. When the handler returns, the CPU goes back to the instruction after `svc`, still **unprivileged**.

`#n` is a small number encoded in the instruction. Real kernels use a request number and arguments to tell which service is wanted; here we only use `svc #0`.

This allows us to implement what effectively are **system calls**.